Secure Coding Review Report

Python Demo Application | Manual Review and Bandit Static Analysis

Prepared by	Keerit Kapoor
Assessment date	26 June 2026
Project scope	Local educational Python demo only
Assessment method	Manual review + Bandit static analysis

Assessment boundary

This review was performed against a local, intentionally insecure demo application. No live systems, external targets, or unauthorised testing were involved. The demo was not executed with untrusted input.

1. Executive Summary

This report documents a secure coding review of a local Python demonstration application created for the CodeAlpha Cyber Security Internship. **The assessment combined manual code inspection with Bandit static analysis.** The review found insecure patterns that would pose material risk if introduced into a production application: hard-coded credentials, weak password hashing, unsafe deserialization, and shell invocation using untrusted command input.

Priority Outcome

4	2 High	6
Priority review findings	Bandit severity count	Total Bandit findings

Scope and Methodology

Scope item	Details
Files reviewed	insecure_app.py and secure_app.py
Static analyzer	Bandit (source-code analysis only)
Manual review focus	Secrets, cryptography, deserialization, command execution
Out of scope	Deployment, network testing, external services, exploit attempts

Ethical and Safety Note

The intentionally insecure file was reviewed but not used as a live service. Bandit reads source code; it does not execute the application functions.

2. Findings Summary

The priority findings below are based on the code review and are consistent with the static-analysis evidence. The Bandit output also includes lower-severity advisory findings; the report focuses on the four patterns that most directly affect application security.

ID	Finding	Risk	Status
SC-01	Hard-coded password	**Medium**	Needs remediation
SC-02	Weak MD5 password hashing	**High**	Needs remediation
SC-03	Unsafe pickle deserialization	**High**	Needs remediation
SC-04	Shell command injection risk	**High**	Needs remediation

Bandit Scan Summary

The scan output captured for this project reports: 2 High, 1 Medium, and 3 Low severity issues. **One confirmed example shown in the output is B602, which flags a subprocess call with shell=True.**

Risk Rating Criteria

Risk	Meaning	Priority action
High	Could enable serious compromise, code execution, or credential exposure	Fix before production
Medium	Could expose secrets or weaken defensive controls	Fix in next development cycle
Low	Advisory or supporting weakness	Track and address during hardening

3. Detailed Findings

SC-01 - Hard-coded Password	MEDIUM

Observation

A database password is stored directly in source code. Secrets committed to repositories can be copied into backups, builds, shared archives, and version history.

Evidence

```
DATABASE_PASSWORD = "demo_password_123"
```

Recommended Remediation

- Load secrets from environment variables or a managed secret store.
- Keep secrets out of version control and rotate any value accidentally committed.
- Use different credentials for development, testing, and production environments.

SC-02 - Weak MD5 Password Hashing	HIGH

Observation

The application uses MD5 to hash passwords. MD5 is fast and not designed for password storage, so it provides weak resistance against password-guessing attacks if hashes are exposed.

Evidence

```
return hashlib.md5(password.encode("utf-8")).hexdigest()
```

Recommended Remediation

- Use a dedicated password-hashing function such as Argon2, bcrypt, scrypt, or PBKDF2-HMAC-SHA256.
- Use a unique random salt for every password hash.
- Use a timing-safe comparison when validating passwords.

3. Detailed Findings (continued)

SC-03 - Unsafe Deserialization with pickle	HIGH

Observation

The demo uses pickle.loads() to process serialized data. Python pickle data is not safe to deserialize from untrusted sources because deserialization can trigger attacker-controlled behavior.

Evidence

```
return pickle.loads(serialized_data)
```

Recommended Remediation

- Use JSON or another data-only interchange format for external data.
- Validate the expected data shape, required fields, and data types after parsing.
- Treat all serialized data received from users, files, queues, or APIs as untrusted.

SC-04 - Shell Command Injection Risk	HIGH

Observation

The function accepts a command string and passes it into subprocess.run() with shell=True. If an attacker controls the input, the shell can interpret unintended command syntax.

Evidence

```
subprocess.run(command, shell=True, text=True, capture_output=True, check=False)
```

Recommended Remediation

- Avoid shell=True when input is not fully controlled.
- Use a fixed allow-list of supported commands and explicit argument lists.
- Set timeouts and handle errors safely.

4. Secure Code Improvements

The project includes secure_app.py, a hardened rewrite intended to demonstrate safer development choices. The table below maps each insecure pattern to the replacement control.

Original risk	Hardened approach	Security benefit
Hard-coded secret	No credential stored in source; use environment/secret management	Reduces accidental secret exposure
MD5 hashing	PBKDF2-HMAC-SHA256 with a random salt and 310,000 iterations	Raises the cost of password guessing
pickle.loads()	json.loads() plus an expected-object check	Avoids unsafe code-capable deserialization
shell=True	Allow-listed command mapping with shell=False, timeout, and check=True	Prevents shell interpretation and limits command execution

Example of Safer Password Handling

```
safe_salt = salt or os.urandom(16)
derived_key = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), safe_salt, ITERATIONS)
```

Example of Safer Command Execution

```
command = ALLOWED_DIAGNOSTICS.get(name)
result = subprocess.run(command, shell=False, timeout=10, check=True, capture_output=True, text=True)
```

5. Static Analysis Evidence

Evidence A - Bandit scan output for insecure_app.py. The static analyzer reads source code without executing the deliberately insecure demo functions. The screenshot confirms the B602 shell=True finding and the reported severity summary.

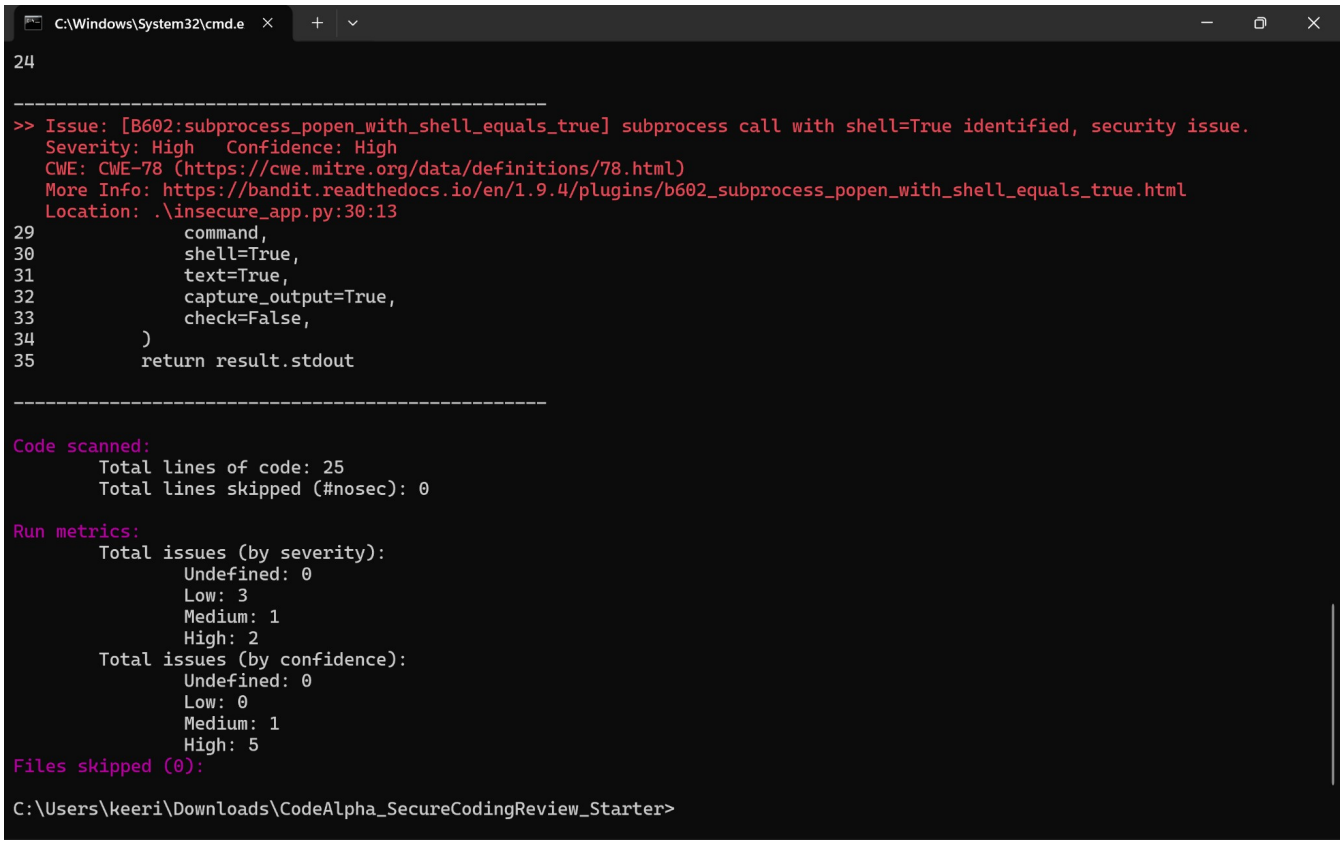

Figure 1. Bandit static-analysis output captured from the local review environment.

Interpretation

- Static analysis identifies patterns that merit developer review; findings should be triaged with code context.
- The B602 finding confirms that shell=True should be removed from the diagnostic function.
- The report combines the tool evidence with manual inspection to prioritise remediation.

6. Remediation Plan and Conclusion

Recommended Remediation Sequence

Priority	Action	Owner	Suggested timing
1	Replace MD5 and pickle usage; remove shell=True	Developer	Before any deployment
2	Remove hard-coded credential and rotate it if previously shared	Developer / DevOps	Immediately
3	Add secret scanning and Bandit to CI checks	Engineering team	Next sprint
4	Review all command execution and data parsing paths	Security reviewer	Before release

Conclusion

The secure coding review identified several common but high-impact weaknesses in a local Python demonstration application. **The most urgent improvements are replacing weak password hashing, eliminating unsafe deserialization, and preventing shell command injection.** The hardened rewrite demonstrates practical secure alternatives that reduce risk before software reaches production.

Skills Demonstrated

- Manual secure code review
- Bandit static-analysis workflow
- Risk classification and remediation planning
- Python secure-coding practices
- Security documentation and GitHub portfolio preparation

References

Bandit static analyzer; OWASP secure coding guidance; Python standard-library documentation for hashlib, json, pickle, and subprocess.

Disclaimer

This report is an educational deliverable for the CodeAlpha Cyber Security Internship. The application under review is a local, intentionally insecure teaching demo. No live systems, user data, or external services were tested.